

```
# Linear Search
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# contoh
data = [10, 20, 30, 40, 50]
print(linear_search(data, 30)) # output: 2
```

2

Best Case : $O(1)$

Average Case : $O(n)$

Worst Case : $O(n)$

1.def linear_search(arr, target): Mendefinisikan fungsi bernama linear_search, memiliki dua parameter:

arr: variabel yang berisi daftar data

target: nilai yang ingin dicari posisinya

2.for i in range(len(arr)):

Melakukan perulangan dari indeks ke-0 sampai indeks terakhir sesuai panjang data (len(arr)). Variabel i berfungsi sebagai penanda posisi/indeks elemen.

3.if arr[i] == target:

Mengecek apakah elemen pada indeks ke-i sama dengan nilai yang dicari.

4.return i

Jika nilainya sama, maka fungsi langsung mengembalikan nilai indeks i, proses pencarian selesai.

5.return -1

Jika seluruh elemen sudah diperiksa dan tidak ada yang sama, fungsi mengembalikan nilai -1 sebagai tanda bahwa data tidak ditemukan.

```
# Binary Search (Iteratif)
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
```

```
        return mid
    elif arr[mid] < target:
        left = mid + 1
    else:
        right = mid - 1

    return -1

# contoh (data harus terurut)
data = [10, 20, 30, 40, 50]
print(binary_search(data, 40)) # output: 3

3
```

Best Case : $O(1)$

Average Case : $O(\log n)$

Worst Case : $O(\log n)$

1. def binary_search(arr, target):

Mendefinisikan fungsi pencarian biner, dengan parameter:

arr: daftar data yang sudah terurut

target: nilai yang ingin dicari posisinya

2. left, right = 0, len(arr) - 1 Menetapkan batas awal dan akhir pencarian:

left = 0: indeks paling kiri/awal

right = len(arr) - 1: indeks paling kanan/akhir

3. while left <= right:

Perulangan berjalan selama batas kiri belum melewati batas kanan (masih ada data yang diperiksa).

4. mid = (left + right) // 2

Menghitung posisi nilai tengah, dibagi dua dan dibulatkan ke bawah. Ini adalah kunci algoritma ini.

5. if arr[mid] == target:

Jika nilai tengah sama dengan yang dicari → langsung kembalikan indeks mid → pencarian selesai.

6. elif arr[mid] < target:

Jika nilai tengah lebih kecil dari target → berarti nilai yang dicari ada di bagian kanan → ubah batas kiri menjadi mid + 1 (buang bagian kiri).

7. else:

Jika nilai tengah lebih besar dari target → berarti nilai yang dicari ada di bagian kiri → ubah batas kanan menjadi $\text{mid} - 1$ (uang bagian kanan).

8. return -1

Jika perulangan selesai dan tidak ditemukan → kembalikan -1 (tanda data tidak ada).

```
# Binary Search (Rekursif)
def binary_search_recursive(arr, left, right, target):
    if left > right:
        return -1

    mid = (left + right) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, mid + 1, right, target)
    else:
        return binary_search_recursive(arr, left, mid - 1, target)

# contoh
data = [10, 20, 30, 40, 50]
print(binary_search_recursive(data, 0, len(data)-1, 20)) # output: 1
```

1

Best Case : $O(1)$

Average Case : $O(\log n)$

Worst Case : $O(\log n)$

1. def binary_search_recursive(arr, left, right, target):

Mendefinisikan fungsi dengan 4 parameter:

arr: daftar data yang sudah terurut

left: batas indeks awal pencarian

right: batas indeks akhir pencarian

target: nilai yang dicari

2. if left > right: return -1

Kondisi Berhenti (Basis Case). Jika batas kiri sudah melewati batas kanan, berarti seluruh data sudah diperiksa dan tidak ditemukan → kembalikan nilai -1.

3. mid = (left + right) // 2

Menghitung posisi nilai tengah, dibagi dua dan dibulatkan ke bawah.

4. if arr[mid] == target: return mid

Jika nilai tengah sama dengan target → langsung kembalikan indeks mid, proses selesai.

5.elif arr[mid] < target:

Jika nilai tengah lebih kecil dari target → nilai yang dicari ada di bagian kanan.

→ Panggil kembali fungsi dengan batas kiri diubah menjadi mid + 1.

6.else:

Jika nilai tengah lebih besar dari target → nilai yang dicari ada di bagian kiri.

→ Panggil kembali fungsi dengan batas kanan diubah menjadi mid - 1.

```
# Jump Search

import math

def jump_search(arr, target):
    n = len(arr)
    step = int(math.sqrt(n))
    prev = 0

    # lompat blok
    while arr[min(step, n) - 1] < target:
        prev = step
        step += int(math.sqrt(n))

    if prev >= n:
        return -1

    # linear search di blok
    for i in range(prev, min(step, n)):
        if arr[i] == target:
            return i

    return -1

# contoh
data = [10, 20, 30, 40, 50, 60, 70]
print(jump_search(data, 60)) # output: 5
```

5

Best Case : $O(1)$

Average Case : $O(n)$

Worst Case : $O(n)$

1.import math

Memanggil pustaka matematika untuk menghitung akar kuadrat (sqrt).

2.n = len(arr)

Menghitung jumlah total elemen dalam data → $n = 7$.

3.step = int(math.sqrt(n))

Menentukan jarak lompatan: $\sqrt{7} \approx 2,64 \rightarrow$ dibulatkan jadi 2. Jadi tiap kali melompat sejauh 2 indeks.

4.prev = 0

Penanda posisi awal blok yang sedang diperiksa.

5.while arr[min(step, n) - 1] < target:

Cek nilai di ujung blok saat ini → arr[step - 1].

Jika nilainya lebih kecil dari target → berarti target ada di blok selanjutnya → geser posisi blok.

min(step, n): Menjaga agar tidak melewati batas akhir data.

6.prev = step & step += int(math.sqrt(n))

Geser awal blok ke akhir blok sebelumnya, lalu tambah jarak lompatan lagi sejauh $\sqrt{n}$.

7.if prev >= n: return -1

Jika posisi awal blok sudah melewati panjang data → data tidak ada.

8.for i in range(prev, min(step, n)):

Setelah blok tempat target berada ditemukan → lakukan pencarian linier satu per satu dari awal blok sampai akhir blok.

9.if arr[i] == target: return i

Jika nilai cocok → kembalikan indeksnya. Jika sampai akhir blok tidak ketemu → kembalikan -1.

```
# Interpolation Search

def interpolation_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high and target >= arr[low] and target <= arr[high]:
        if low == high:
            if arr[low] == target:
                return low
            return -1

        # rumus interpolasi
        pos = low + ((target - arr[low]) * (high - low) //
                    (arr[high] - arr[low]))
```

```

        if arr[pos] == target:
            return pos
        elif arr[pos] < target:
            low = pos + 1
        else:
            high = pos - 1

    return -1

# contoh
data = [10, 20, 30, 40, 50]
print(interpolation_search(data, 50)) # output: 4

4

```

Best Case : $O(1)$

Average Case : $O(\log n)$

Worst Case : $O(n)$

1. $low = 0$ & $high = len(arr) - 1$

Menetapkan batas awal dan akhir pencarian.

2. while $low \leq high$ and $target \geq arr[low]$ and $target \leq arr[high]$:

Kondisi berjalan hanya jika:

Masih ada rentang data ($low \leq high$)

Nilai target berada dalam rentang nilai data yang ada. Jika target lebih kecil dari nilai terkecil atau lebih besar dari nilai terbesar $\rightarrow$ langsung berhenti.

3. if $low == high$:

Jika tersisa satu elemen, langsung cek saja. Jika sama $\rightarrow$ kembalikan indeks, jika tidak $\rightarrow -1$.

4. $pos = low + ((target - arr[low]) * (high - low)) // (arr[high] - arr[low])$

Inti algoritma: menghitung posisi tebakan berdasarkan nilai data.

Pada contoh: $arr = [10, 20, 30, 40, 50]$, cari 50

$\rightarrow pos = 0 + ((50 - 10) * (4 - 0)) // (50 - 10)$

$\rightarrow pos = 0 + (40 * 4) // 40 = 0 + 160 // 40 = 4$

5. if $arr[pos] == target$:

Jika nilai di posisi hitungan sama $\rightarrow$ langsung ketemu.

6. elif $arr[pos] < target$:

Nilai tebakan terlalu kecil $\rightarrow$ geser batas bawah ke $pos + 1$ (cari ke kanan).

7. else:

Nilai tebakan terlalu besar → geser batas atas ke pos - 1 (cari ke kiri).

8.return -1

Jika keluar dari perulangan → data tidak ada.